

Setup

BuMoChi

Quick start: one-command pipeline launcher

The launcher starts and supervises **Bunraku0SCEncoder** and **Bunraku0SCDecoder**. With no arguments it starts the local-only pipeline and does not start **OscGroupClient**:

```
./PipelineApplications/start_bumochi_pipeline.sh
```

Run that command from the root of the BuMoChi repository. It uses the standard local ports automatically:

```
XR-Animator -> encoder 39537 -> Bmc 57130 -> decoder 39538 -> Godot 39539
```

The launcher does not start XR-Animator, SuperCollider, or Godot. Start those three applications separately. Recompile the SuperCollider class library so that Bmc listens on **57130**, configure XR-Animator to send to **39537**, and run a Godot avatar receiver on **39539**.

To include OSCGroups, supply an OSC server address and a unique username:

```
./PipelineApplications/start_bumochi_pipeline.sh \
  --oscservice SERVER_ADDRESS \
  --username PerformerA
```

OSCGroups mode uses these additional defaults:

Setting	Default	Override
Group name	bumochi	--groupname NAME
User and group password	bmc123	--password PASSWORD
Server port	22242	--server-port PORT
Local network-facing port	22243	--local-port PORT

The password **bmc123** is a convenient rehearsal default, not a secure public-network credential. Set a different password for an exposed OSCGroups server.

Inspect every available launcher option without starting anything:

```
./PipelineApplications/start_bumochi_pipeline.sh --help
```

Stop all helper processes started by the launcher with **Control-C** in its terminal.

When the launcher is started with **--verbose**, follow the live encoder and decoder output in another terminal:

```
tail -f "$TMPDIR/bumochi-pipeline-$USER/encoder.log" \
        "$TMPDIR/bumochi-pipeline-$USER/decoder.log"
```

The launcher starts Python in unbuffered mode, so new log lines appear immediately.

Cheatsheet! Check the active Bmc port configuration

Your VMC mocap applications (XR-Animator, Waidayo, or other) and Godot should be configured to send/receive animation data at the ports required by the pipeline. To check these port numbers, after recompiling the SuperCollider class library, evaluate:

```
Bmc.help;
```

The default summary is equivalent to:

XR-Animator output port: 39537
 BunrakuOSCEncoder output port: 57130
 SuperCollider VMC/OSC input port: 57130
 SuperCollider VMC/OSC output port: 39538
 BunrakuOSCDecoder input port: 39538
 BunrakuOSCDecoder output ports:
 Ishidomaru: 39539

The final list is generated from the avatars currently configured in Bmc, so it expands when additional avatar routes are added. Run **Bmc.help**; again after changing any ports.

The following sections are only necessary if you want to customize the pipeline or check for errors.

Details

This guide provides information to troubleshoot or tweak the internal port numbers of all applications of the pipeline. It shows how to start and configure the six applications in the following downstream order:

No.	Application	Input/listening port	Output/destination port	Required configuration
1	XR-Animator	—	39537	Send standard VMC to BunrakuOSCEncoder on 127.0.0.1:39537 .
2	BunrakuOSCEncoder	39537	57130; optionally 22244	Listen for XR-Animator; always send route-free frames to local Bmc on 57130 ; in OSC-Groups mode, send an identical copy to local OscGroupClient on 22244 .
3	OscGroupClient	22244	57130	Receive local encoder frames on 22244 ; deliver remote collaborators' frames to local Bmc on 57130 . The client also connects to OscGroupServer port 22242 through a client-specific local port such as 22243 .
4	SuperCollider / Bmc	57130	39538	Receive local and remote route-free frames; synthesize the local scene; send routed frames to BunrakuOSCDecoder on 39538 . Ishidomaru's default routed frames embed Godot destination 39539 .
5	BunrakuOSCDecoder	39538	39539	Receive routed Bmc frames, reconstruct standard VMC, and forward Ishidomaru to the embedded Godot port 39539 .
6	Godot	39539	—	Run an Ishidomaru VMC receiver listening on 39539 and render the animation locally.

The resulting application and port sequence is:

```

1. XR-Animator      --VMC 39537-----> 2. BunrakuOSCEncoder
2. BunrakuOSCEncoder --frames 57130-----> 4. SuperCollider / Bmc
2. BunrakuOSCEncoder --frames 22244-----> 3. OscGroupClient
3. OscGroupClient   --remote frames 57130-> 4. SuperCollider / Bmc
4. SuperCollider / Bmc --routed frames 39538-> 5. BunrakuOSCDecoder
5. BunrakuOSCDecoder --VMC 39539-----> 6. Godot / Ishidomaru
  
```

The procedure uses the default single-avatar Ishidomaru configuration. Ishidomaru's Godot VMC receiver listens on **39539**.

Run all terminal commands from the root of the BuMoChi repository unless the command uses a globally installed launcher.

Only the receiving application owns a UDP listening port. Several applications may send packets to Bmc on **57130** without conflict because Bmc is the single listener.

Before starting

1. Confirm that Python 3 is available:

```
python3 --version
```

2. For OSCGroups collaboration only, obtain the **OscGroupServer** address and a unique username from the session organiser. Confirm whether the session uses the default group **bumochi** and password **bmc123** or overrides them.

3. For OSCGroups collaboration only, give this workstation a unique OSCGroups username and encoder **--source** identity. The launcher uses local network-facing port **22243** unless it is overridden.
4. Stop older encoder, decoder, and **OscGroupClient** processes. If needed, inspect the local receiving ports:

```
lsof -nP -iUDP:39537
lsof -nP -iUDP:22244
lsof -nP -iUDP:57130
lsof -nP -iUDP:39538
lsof -nP -iUDP:39539
```

1. XR-Animator

1. Start XR-Animator.
2. Open its VMC/OSC output settings.
3. Set the VMC destination to:

Host: 127.0.0.1
Port: 39537

4. Enable full-body VMC output.

XR-Animator may start before the encoder. UDP frames sent before the encoder begins listening are simply discarded.

For screenshots and detailed configuration instructions, see [XR-Animator VMC output port](#) and [Configure XR-Animator](#).

2. BunrakuOSCEncoder

Open a Terminal window at the BuMoChi repository root and start the encoder with every relevant port written explicitly:

```
python3 PipelineApplications/BunrakuOSCEncoder.py \
  --listen-ip 127.0.0.1 \
  --listen-port 39537 \
  --bmc-ip 127.0.0.1 \
  --bmc-port 57130 \
  --oscggroups-ip 127.0.0.1 \
  --oscggroups-port 22244 \
  --avatar "Ishidomaru" \
  --source "workstation-a-xr-animator" \
  --verbose
```

Replace **workstation-a-xr-animator** with a stable source identity unique to this workstation. Leave this terminal open.

The encoder produces two identical route-free source-frame copies:

```
local copy    -> UDP 57130 -> local Bmc
network copy  -> UDP 22244 -> local OscGroupClient
```

Because this startup order places the encoder before **OscGroupClient** and Bmc, its first outgoing packets may be discarded. Normal delivery begins as soon as those receivers start.

If the globally installed launcher is available, the equivalent command is:

```
BunrakuOSCEncoder \
  --listen-ip 127.0.0.1 \
  --listen-port 39537 \
  --bmc-ip 127.0.0.1 \
  --bmc-port 57130 \
  --oscggroups-ip 127.0.0.1 \
  --oscggroups-port 22244 \
  --avatar "Ishidomaru" \
  --source "workstation-a-xr-animator" \
```

--verbose

For installation, options, and diagnostic instructions, see [How to start BunrakuOSCEncoder](#) and [Test and diagnose BunrakuOSCEncoder](#).

3. OscGroupClient

Open a second Terminal window at the BuMoChi repository root.

OscGroupClient requires nine positional arguments in this order:

```
OscGroupClient SERVER_ADDRESS SERVER_PORT LOCAL_TO_REMOTE_PORT INPUT_PORT OUTPUT_PORT  
↪ USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

Start the bundled macOS client with:

```
HelperAppsAndExamples/OSCGroups/bin/macos/OscGroupClient \  
  SERVER_ADDRESS 22242 22243 22244 57130 \  
  USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

Replace all uppercase placeholders. Replace example local port **22243** if it is occupied or if another client on this computer uses it. Leave this terminal open.

The important application-facing ports are:

```
OscGroupClient input:  UDP 22244 <- local encoder frames for sharing  
OscGroupClient output: UDP 57130 -> remote source frames delivered to local Bmc
```

Do not set the client output to decoder port **39538**. Remote route-free frames must enter Bmc first for local selection, composition, recording, and synthesis.

If the globally installed client is available, use the same arguments:

```
OscGroupClient \  
  SERVER_ADDRESS 22242 22243 22244 57130 \  
  USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

For argument definitions, installation, and checks, see [How to start OscGroupClient](#), [OscGroupClient command-line arguments](#), and [OSCGroupClient port](#).

4. SuperCollider

1. Start SuperCollider.
2. Recompile the class library if necessary with **Language → Recompile Class Library**.
3. After compilation, Bmc automatically listens for **/bunraku/vmc/frame** on UDP **57130**.
4. Evaluate this block to verify and explicitly restore the default Ishidomaru route:

```
(  
Bmc.avatar(\Ishidomaru).vmcPort_(39539);  
Bmc.decoderPort_(39538);  
Bmc.forwardDecoder_(true);  
Bmc.start(57130);  
Bmc.status;  
)
```

5. Open the live input monitor:

```
Bmc.showDispatcherStatus;
```

Its static field should say:

```
Listening for '/bunraku/vmc/frame' on port: 57130
```

The dynamic **received** count should increase while XR-Animator is sending. It may include both local frames received directly from the encoder and remote frames received through **OscGroupClient**.

For details, see [SuperCollider port listening to OSCGroupClient](#), [Default forwarding pipeline of BuMoChi](#), and [Bmc system control](#).

5. BunrakuOSCDecoder

Open a third Terminal window at the BuMoChi repository root and start the decoder:

```
python3 PipelineApplications/BunrakuOSCDecoder.py \  
--listen-ip 127.0.0.1 \  
--listen-port 39538 \  
--target-ip 127.0.0.1 \  
--allow-target-port 39539 \  
--verbose
```

Leave this terminal open. The decoder receives locally synthesized routed frames from Bmc on **39538**, reads the embedded avatar destination, reconstructs standard VMC bundles, and sends Ishidomaru's bundles to Godot on **39539**.

If the globally installed launcher is available, the equivalent command is:

```
BunrakuOSCDecoder \  
--listen-ip 127.0.0.1 \  
--listen-port 39538 \  
--target-ip 127.0.0.1 \  
--allow-target-port 39539 \  
--verbose
```

For a multi-avatar Godot scene, permit every destination used by that scene. For example:

```
python3 PipelineApplications/BunrakuOSCDecoder.py \  
--listen-ip 127.0.0.1 \  
--listen-port 39538 \  
--target-ip 127.0.0.1 \  
--allow-target-port 39539 \  
--allow-target-port 39540 \  
--verbose
```

For installation, options, and diagnostics, see [How to start BunrakuOSCDecoder](#) and [Test and diagnose BunrakuOSCDecoder](#).

6. Godot

1. Start Godot and open the intended avatar project.
2. For the default single-avatar test, open **Seed_2_Ishidomaru_C**.
3. Confirm that the project's Ishidomaru VMC receiver listens on UDP **39539**.
4. Run the project scene.

Godot may start last. VMC bundles sent before its receiver starts are discarded; live animation begins when the scene is running and listening.

For detailed inspection and configuration instructions, see [Godot avatar-specific listening port](#).

For the uniform two-avatar E project, use two explicit receiver nodes:

Avatar	Receiver node	UDP port	Body tracker name	Face tracker name
Mother	MotherVMCTracker	39539	/vmc/mother_body_tracker	/vmc/mother_face_tracker
Ishidomaru	IshidomaruVMCTracker	39540	/vmc/ishidomaru_body_tracker	/vmc/ishidomaru_face_tracker

When using that scene, also set Bmc's avatar routes accordingly and start the decoder with both allowed ports:

```
Bmc.avatar(\Ishidomaru).vmcPort_(39540);  
Bmc.addAvatar(\Mother, "Mother").vmcPort_(39539);
```

See [Multi-Avatar project port number setting in Godot](#) for the explicit-receiver arrangement.

Verify the complete pipeline

1. XR-Animator reports that VMC output is enabled for **127.0.0.1:39537**.

2. The encoder's **received**, **bmc_sent**, and **oscggroups_sent** counters increase.
3. **OscGroupClient** reports successful server and group registration.
4. **Bmc.showDispatcherStatus** reports **running: true**, port **57130**, and an increasing **received** count.
5. The decoder reports incoming frames on **39538** and VMC output to **39539**.
6. Ishidomaru moves in Godot.

To test only the final SuperCollider-to-Godot path, evaluate:

```
Bmc.sendCalibrationFrame;
```

Shutdown

Stop the applications in reverse order:

1. Stop the running Godot scene.
2. Stop **BunrakuOSCDecoder** with **Control-C**.
3. Evaluate **Bmc.stop;** in SuperCollider.
4. Stop **OscGroupClient** with **Control-C**.
5. Stop **BunrakuOSCEncoder** with **Control-C**.
6. Disable XR-Animator VMC output or quit XR-Animator.